

gods_eye_v1_approach_3 — LangGraph State Machine Architecture with Structured Memory

Title

gods_eye_v1_approach_3: Graph-Orchestrated Agents with LangGraph State Machine, Qdrant Persistence, and Approval Gates

Design Period: Conceptual Iteration 3

Status: Deprecated — Superseded by Approach 4

Predecessor: Approach 2 (Multi-Agent Message Bus)

Problem Statement

Approach 2 demonstrated that typed message passing is the wrong coordination model for autonomous agents: messages require receivers to be ready, create ordering ambiguities, and make human-in-the-loop interactions structurally difficult. The core insight coming out of Approach 2 is that **coordination should be expressed as state transitions, not as message delivery**.

Approach 3 adopts LangGraph as the orchestration framework. LangGraph models the agent workflow as a directed graph where nodes are agent actions and edges are conditional transitions based on state. All agents write to and read from a shared `TypedDict` state object that LangGraph manages. This eliminates the message bus — coordination happens through state mutations, not message emissions.

Additionally, Approach 3 introduces:

- A user **approval gate** implemented via LangGraph's `interrupt_before` mechanism, which pauses graph execution at designated nodes pending user confirmation.
- **Qdrant** (replacing Chroma) as the vector store, deployed as a standalone server to support concurrent multi-client writes.
- A dedicated **AnalystNode** for structured intelligence extraction — the first iteration to separate finding extraction from finding storage.
- **Metasploit RPC API** integration for post-exploitation tasks that require interactive session management.

The structural problem of this iteration is subtler than Approach 2's: **LangGraph's state machine model imposes a graph topology that must be defined at design time**. The nodes and edges of the graph encode the agent workflow. When the LLM's replanning decisions require adding new nodes, modifying edges, or branching in ways not anticipated in the static graph definition, LangGraph has no mechanism to support this — the graph is fixed at compile time.

The system can route between pre-defined nodes, but it cannot autonomously restructure its own execution graph.

This is the **static topology trap**: the architecture is expressive enough to handle anticipated workflow variations (if-else branching between exploitation techniques) but cannot handle genuinely novel discoveries (a new attack surface that requires an entirely new task subtree). The PDF ultimately rejects the graph-as-orchestration model in favor of a DAG-as-data model, where the task graph lives in state.json as a data structure that the Supervisor can rewrite at runtime.

Proposed Architecture

Overview

The system is defined as a LangGraph `StateGraph` with six node types:

1. **PlannerNode** — LLM-driven task decomposition. Writes task list to graph state.
2. **DispatcherNode** — Reads task list, identifies ready tasks, routes to `ExecutorNode` or `TerminalNode`.
3. **ExecutorNode** — Executes a single task via shell commands. Writes raw output to graph state.
4. **ApprovalNode** — LangGraph interrupt node. Pauses graph execution, surfaces command to user, resumes on approval.
5. **AnalystNode** — Consumes `ExecutorNode` output, extracts structured findings, writes to Qdrant and Mem0.
6. **ReplannerNode** — Activated on task failure. Reads findings, modifies task list in graph state, routes back to `DispatcherNode`.

Edges in the graph:

- `PlannerNode → DispatcherNode` (always)
- `DispatcherNode → ExecutorNode` (if tasks pending)
- `DispatcherNode → TerminalNode` (if all tasks done/skipped)
- `ExecutorNode → ApprovalNode` (before each command)
- `ApprovalNode → ExecutorNode` (on approval, back to reasoning)
- `ApprovalNode → ExecutorNode` (on denial, with denial injected into state)
- `ExecutorNode → AnalystNode` (on task completion or failure)
- `AnalystNode → DispatcherNode` (on task success, trigger next dispatch)
- `AnalystNode → ReplannerNode` (on task failure, trigger replan)
- `ReplannerNode → DispatcherNode` (after replan, check for newly ready tasks)

Component Breakdown

LangGraph (langgraph) The orchestration backbone. `StateGraph` with a `TypedDict` state schema. The state schema includes: `goal: str`, `tasks: Dict[str, TaskSpec]`, `nuances: Dict[str, str]`, `current_task_id: str`, `current_command: str`, `current_output: str`, `findings: List[FindingSpec]`, `replan_count: int`.

Selection rationale: LangGraph's state-machine model directly solves Approach 2's message ordering problem — there is no bus, no queue, no delivery guarantee to reason about. The state object is the single source of truth, and all agents read from and write to it atomically (within a single graph step).

Limitations: LangGraph's `interrupt_before` mechanism for the `ApprovalNode` is synchronous in the default execution model — it blocks the entire graph runner process while waiting for user input. This prevents any other graph nodes from executing during approval wait time. In a multi-task scenario, this means every pending approval blocks all other task progress, creating a serial approval bottleneck.

LangGraph `interrupt_before` (Approval Gate) The approval gate is implemented by designating `ExecutorNode` with `interrupt_before=True`. When the graph reaches `ExecutorNode` with a proposed command in state, LangGraph serializes the graph state to disk and pauses. The user reviews the command and inputs approval/denial. LangGraph deserializes state and resumes.

Limitations: State serialization/deserialization adds latency. More critically, the `interrupt_before` mechanism operates at node granularity — it cannot interrupt within a node. If `ExecutorNode` proposes a command, the interruption happens before the node executes, not before the command runs. This means the LLM reasoning step and command proposal happen in the same node invocation — the user sees the proposed command but cannot inspect the intermediate reasoning that produced it.

Qdrant (qdrant-client, standalone server) Replaces Chroma as the vector store. Qdrant is deployed as a Docker container with a named volume for persistence. The `qdrant-client` Python library is used for all operations. `AnalystNode` writes finding vectors with payload fields: `task_id`, `severity`, `cve_ids`, `tags`, `raw_output_hash` (for deduplication).

Selection rationale: Qdrant's server architecture supports concurrent clients, solving Chroma's write contention problem. Qdrant's filtering API (filter by `severity`, `task_id`, `tags`) is significantly more expressive than Chroma's metadata filtering.

Limitations: Qdrant does not have a native upsert-by-semantic-similarity primitive — deduplication still requires a query-before-write pattern. The `AnalystNode` implements this as: query Qdrant for top-1 similar finding; if cosine similarity > 0.95, skip write. This threshold is empirically tuned but fragile — slightly differently phrased descriptions of the same CVE can fall below the threshold and produce duplicates.

Mem0 (mem0ai) Entity graph memory introduced in this iteration. `AnalystNode` writes typed entities (Host, Service, Vulnerability, Credential) to Mem0 with relationship annotations. Mem0 is queried by `PlannerNode` and `ExecutorNode` at session start to load known target context.

Selection rationale: Mem0 provides a structured entity graph with relationship-aware retrieval — querying "services running on host 10.0.0.5" returns a properly related set of Service entities, not a flat list of documents. This is qualitatively different from vector similarity search and complements Qdrant's semantic search capability.

Limitations: At this iteration, Mem0 writes are synchronous and blocking within AnalystNode. Writing a large batch of entities (e.g., all open ports from an nmap scan) can take several seconds, stalling the graph at the AnalystNode step.

Metasploit RPC API (pymetasploit3) Post-exploitation tool integrated into ExecutorNode.

`pymetasploit3` connects to a locally running `msfrpcd` daemon and exposes Metasploit modules, sessions, and console interfaces as Python calls. This allows the Executor to run Metasploit modules without subprocess-level CLI interaction.

Selection rationale: Subprocess-level Metasploit interaction (sending commands to `msfconsole` via `pexpect`) is fragile and stateful in ways that are hard to manage. The RPC API provides a structured interface: `client.modules.use('exploit', 'multi/handler')`, `client.sessions.session(sid).run_with_output('whoami')`.

Limitations: `pymetasploit3` requires a running `msfrpcd` with known credentials, adding a daemon management dependency. The RPC API exposes only a subset of Metasploit's capability — complex workflows requiring interactive console interaction still require subprocess. Session management (keeping Meterpreter sessions alive, handling session drops) is not handled by the architecture and must be manually managed.

LangChain Tool Definitions (for AnalystNode LLM) AnalystNode uses a LangChain-structured LLM call with a strict Pydantic output schema for finding extraction. The schema enforces fields: `finding_type: Literal['vulnerability', 'credential', 'service', 'network']`, `severity: Literal['critical', 'high', 'medium', 'low', 'info']`, `description: str`, `evidence: str`, `cve_ids: List[str]`, `affected_asset: str`. This structured extraction replaces the regex-based finding extraction of earlier approaches.

Docker SDK (docker-py) ExecutorNode uses Docker SDK for optional sandboxed command execution. A persistent Kali Linux container is maintained per engagement session. Commands are executed via `container.exec_run()` with stream output support. Stream output is collected and stored as full raw text in graph state (no truncation).

Limitations: `container.exec_run()` returns a blocking generator when `stream=True`. The generator must be fully consumed before the node returns, which means long-running commands (nmap, gobuster) block the LangGraph execution step for their entire duration. This is the same single-threaded execution trap as Approach 1, now at the LangGraph node level rather than the subprocess level.

Failure Points

1. Static Graph Topology Cannot Represent Dynamic Task Structures

The most fundamental architectural failure of this approach. LangGraph requires the graph topology (nodes and edges) to be defined at initialization time. The PlannerNode's task decomposition is encoded as data in the state object, but the execution graph itself is static. When a task produces a finding that requires a genuinely novel subtask type — for example, discovering an LDAP server mid-engagement that requires a specialized LDAP enumeration subtask — the graph has no LDAP-specific node to route to. The DispatcherNode can only route to pre-defined ExecutorNode variants.

In practice, this forces the design to anticipate all possible task types at graph design time, which defeats the purpose of LLM-driven planning. The workaround — making ExecutorNode a universal "do anything" node — collapses the graph's structure back toward the monolithic Approach 1 model.

2. interrupt_before Creates a Serial Approval Bottleneck

The LangGraph `interrupt_before` mechanism pauses the entire graph runner, not just the active subgraph branch. In a multi-task scenario where two ExecutorNode instances are running concurrently (LangGraph supports subgraph parallelism), a pending approval for Task A blocks Task B's progress even if Task B's commands are ready to execute.

More critically: if the user is unavailable (not watching the terminal), the entire system halts waiting for approval. There is no timeout on the approval gate, no way to route around a pending approval, and no way for the user to pre-approve categories of commands. The approval mechanism solves the control problem but at the cost of **parallelism and resilience**.

3. LangGraph State Object Size Grows Unboundedly

The shared state `TypedDict` stores `current_output: str` for the most recent command. For long engagements with many tasks, the cumulative `findings: List[FindingSpec]` list grows without bound in the state object. LangGraph serializes the entire state object on every checkpoint (for `interrupt_before` recovery). A state object with 200 findings and raw outputs grows to several MB, causing checkpoint serialization latency that compounds over time.

More critically: the state object is loaded into memory at the start of every graph step. As the state grows, memory pressure increases monotonically throughout the engagement — there is no mechanism to evict stale state data to external storage while keeping the object addressable.

4. AnalystNode Structural Extraction Is Blocking and Unbatched

AnalystNode processes one `TASK_RESULT` at a time, synchronously. In a parallel task scenario where three ExecutorNode instances complete simultaneously, their outputs queue up for sequential AnalystNode processing. Each AnalystNode invocation involves: one LLM call for structured extraction, one Mem0 write (blocking), one Qdrant write (non-blocking but sequential), and one deduplication query. Total latency per task output: 3-8 seconds. For three

simultaneous completions, the Planner waits 9-24 seconds before receiving all three finding summaries.

This is a **sequential analysis bottleneck**: the architecture has parallel execution but serial intelligence extraction, which negates much of the parallelism benefit.

5. ReplannerNode Has No Access to External Intelligence

The ReplannerNode is invoked on task failure and reads the current graph state (findings, nuances, failure reason) to make a replanning decision. However, the ReplannerNode has no tool to query ExploitDB or SearxNG — it replans based only on information already in the graph state. If a task fails because the attack technique requires a specific tool flag or a recently-published bypass, the Replanner cannot discover this and will replan using outdated or incomplete knowledge.

This is a **knowledge isolation failure**: the replanning agent is cut off from external intelligence at precisely the moment it most needs current exploit information.

6. ExecutorNode Holds Full Task Context in LangGraph State

When Dispatcher routes to ExecutorNode, the full task context (goal, nuances, relevant Qdrant findings, Mem0 entities) is stored in the LangGraph state object as a serialized string. For complex tasks with many prior findings, this context string can be thousands of tokens. LangGraph does not have a mechanism to lazy-load this context — it is always fully serialized with the state checkpoint, inflating checkpoint size and increasing deserialization latency on ApprovalNode resume.

7. Metasploit RPC Session Management Is External to the Architecture

When ExecutorNode establishes a Meterpreter session via Metasploit RPC, the session is a stateful object managed by the `(msfrpcd)` daemon. The LangGraph state object stores only the session ID. If the session drops between two ExecutorNode invocations (network interruption, target reboot, timeout), the ExecutorNode discovers the drop at execution time — not at planning time. The PlannerNode has no mechanism to detect session liveness or incorporate session state into its task dependency model. Tasks that depend on an active session will fail silently until ExecutorNode attempts to use the dead session.

Comparison to Approach 2

Approach 3 makes significant structural improvements over Approach 2:

Problem in Approach 2	How Approach 3 Addresses It
Message ordering inconsistencies	LangGraph state machine eliminates message bus entirely
Swarm context collision	LangGraph state object provides atomic shared state
Chroma write contention	Qdrant server with concurrent client support
No approval gate	LangGraph <code>interrupt_before</code> mechanism
No structured finding extraction	Dedicated AnalystNode with Pydantic schema
Chroma deduplication failure	Qdrant cosine-similarity deduplication in AnalystNode
No Mem0 entity graph	Mem0 introduced for typed entity storage

However, Approach 3 introduces the static topology trap, the serial approval bottleneck, and the replanner knowledge isolation failure — all of which are architecturally more difficult to solve within the LangGraph model than by replacing the model entirely.

Why This Is NOT the Final PDF Architecture

Approach 3 is the first iteration that feels close to a production architecture — it has role separation, persistent memory, approval gates, and structured intelligence extraction. The gap between Approach 3 and the PDF is not in individual components but in the **fundamental coordination model**:

1. **LangGraph's graph is topology, not data.** The PDF represents the task graph as a data structure in `state.json` that the Supervisor can rewrite at runtime. LangGraph's graph is code — it cannot be rewritten at runtime by the LLM.
2. **The approval gate serializes parallelism.** The PDF's `asyncio.Event` per command approval allows multiple Hacker instances to independently wait for their own approvals without blocking each other. LangGraph's `interrupt_before` pauses the entire graph.
3. **There is no Reporter equivalent.** Approach 3 has no dedicated user-facing agent. User queries about current state require pausing or introspecting the LangGraph runner — there is no clean non-blocking interface for the user to query progress during execution.
4. **The Replanner has no Searcher.** The PDF's Supervisor can call the Searcher agent before making a replan decision. This is architecturally critical: replanning decisions informed by current CVE data are qualitatively better than decisions based only on in-state findings.
5. **The AnalystNode bottleneck.** The PDF's Analyst processes each command output asynchronously as it arrives, in parallel with the Hacker's continued reasoning. In Approach 3, the AnalystNode is a blocking graph step — the Hacker must wait for the Analyst to complete before the graph can advance.

What the PDF Fixes Over This Approach

Failure in Approach 3	PDF Solution
Static graph topology	Task DAG as runtime-mutable data in state.json; Supervisor rewrites it on replan
Serial approval bottleneck	Per-command <code>asyncio.Event</code> in HackerAgent; each Hacker blocks only on its own approval
State object unbounded growth	state.json + Mem0 + Qdrant as separate stores; state.json contains only compact task metadata
AnalystNode sequential bottleneck	Analyst coroutine runs asynchronously per Hacker output; multiple Analysts can run in parallel
Replanner has no external intelligence	Supervisor calls SearcherAgent before replan decision; Searcher returns structured ExploitDB intel
No dedicated user interface agent	ReporterAgent runs as a separate always-on coroutine; non-blocking query answering from shared memory
Metasploit session state external to architecture	Hacker owns session management within task context; session state is a nuance in state.json
LangGraph step-level blocking on long commands	Hacker command loop is a free-running asyncio coroutine; no graph step boundary to block